

BBT: BPE-Guided Byte Transformer

Hyunho Kim
Kakao Corp
khe1.2h@kakaocorp.com

SeungYeol Baek
AgileSoDA
sybaek1005@gmail.com

Abstract

Byte Pair Encoding (BPE) tokenizers shorten language model input sequences, while standard BPE Transformers typically build token representations through vocabulary lookup. Under this lookup, each token gets an independent vector that ignores byte-level structure shared across tokens, and in small-scale models the vocabulary-sized token embedding / LM head matrix takes a substantial fraction of parameters. To address this, we propose **BPE-Guided Byte Transformer (BBT)**, a model that uses BPE for segmentation while replacing vocabulary lookup with byte-level encoding and decoding. BBT uses a ByteEncoder to extract BPE-boundary segment representations, a SegmentTransformer to model them causally, and a ByteDecoder to generate the next segment byte by byte, preserving BPE’s sequence compression while removing the vocabulary-sized token embedding / LM head matrix. On a subset of English FineWeb, BBT consistently lowers test bits per byte (BPB) over matched standard BPE Transformers while using 20–40% fewer parameters at comparable FLOPs. Auxiliary evaluations show improved completion at larger hidden sizes, improved corruption robustness, and stronger transfer to unseen languages in a multilingual setting.

1 Introduction

Byte Pair Encoding (BPE) tokenizers are now standard segmentation methods for language models, turning raw byte sequences into shorter sequences of subword units (Sennrich et al., 2016; Kudo and Richardson, 2018). The BPE tokenizer also defines the vocabulary used by a standard BPE Transformer: each token id is mapped through a learned embedding, and the LM head predicts over the same vocabulary. The BPE tokenizer thus plays two roles. First, it segments the byte sequence. Second, it defines both the index space for input

vocabulary lookup¹ and the prediction space for the output LM head.

Of these two roles, the benefit of segmentation is well established because shorter sequences reduce global self-attention cost. However, the decision to reuse the BPE vocabulary as both the input embedding index space and the output LM head prediction space has rarely been examined in isolation.

In terms of representation, each token is assigned independent embedding and LM head rows, so byte-level structure shared across tokens is not reflected in vocabulary-related parameters, and rare tokens must be learned from their own sparse occurrences rather than inherited from more frequent ones (Bojanowski et al., 2017; Pinter et al., 2017). In terms of parameters, since embedding and LM head sizes scale with BPE vocabulary size, a large vocabulary can dominate the budget at small model scales. With the LLaMA 3.2 1B config, the 128K-token embedding matrix alone has about 262.7M parameters (hidden size 2,048), or 21.3% of the 1.23B total (Grattafiori et al., 2024; Meta AI, 2024).

A related line of work removes the fixed subword vocabulary by operating directly on bytes or characters (Xue et al., 2022; Clark et al., 2022; Tay et al., 2022). This replaces the vocabulary-sized token embedding / LM head matrix with much smaller byte- or character-level layers. It also improves robustness to out-of-vocabulary inputs and fine-grained string structure, but byte/character sequences are much longer than token sequences, increasing global transformer cost.

Hierarchical architectures reduce this long-sequence cost by combining local byte-level computation with global patch- or word-level computation (Yu et al., 2023; Slagle, 2024; Pagnoni et al., 2024; Neitemeier et al., 2025). Most of these methods do not rely on a tokenizer; instead, they derive

¹Vocabulary lookup refers to the input-side operation in which the model retrieves a learned vector from the row of the embedding matrix indexed by a BPE token id.

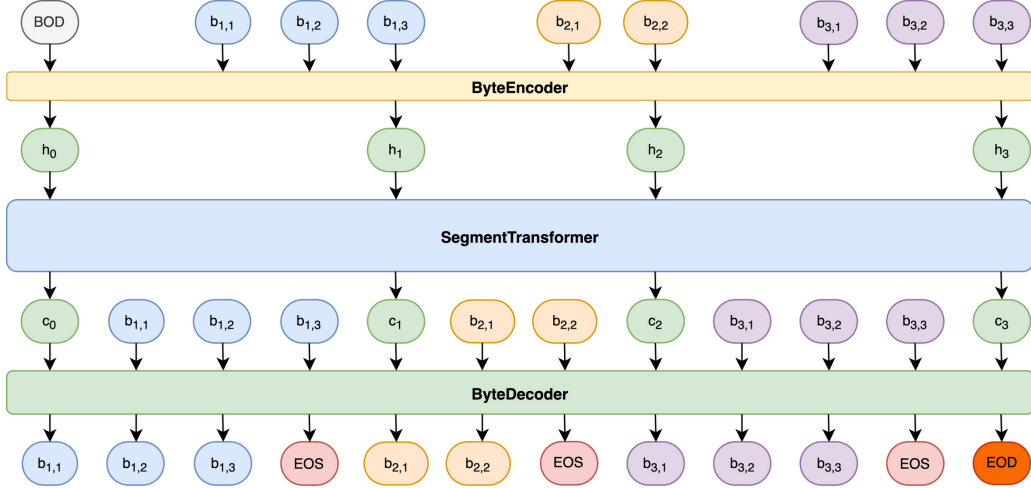


Figure 1: Training architecture of BBT. (1) The byte encoding stage processes the input byte sequence and gathers hidden states at BPE segment boundary positions to form segment representations h_i . (2) The segment modeling stage performs causal self-attention over the segment representation sequence to compute context representations c_i , where each c_i conditions generation of the bytes in the next segment s_{i+1} . (3) The byte decoding stage predicts the next byte, EOS, and EOD from inputs that interleave context vectors with teacher-forced byte sequences. Here, BOD, EOS, and EOD denote the beginning-of-document, end-of-segment, and end-of-document tokens, respectively.

patch boundaries from fixed lengths, surface-form heuristics, or learned/dynamic rules. Our approach instead uses the BPE tokenizer as a source of segment boundaries over bytes, reusing the corpus statistics encoded in the BPE merge table.

We propose BPE-Guided Byte Transformer (BBT) to separate the segmentation and vocabulary lookup roles of the BPE tokenizer. BBT does not use token ids for the embedding or output LM head; it uses only the segment boundaries from the BPE tokenizer, gathering byte-level hidden states at these boundaries (Section 2).

We validate BBT through controlled FineWeb (Penedo et al., 2024) pretraining comparisons with standard BPE Transformers whose segment-level Transformer stacks are matched. This comparison isolates the input/output vocabulary interface under a fixed BPE tokenizer: the data, training schedule, and segment-level transformer body are held constant, while the standard vocabulary lookup and LM head are replaced with BBT’s byte-level interface. In contrast to tokenizer-free hierarchical models, which change both the interface and the induced segment boundaries, BBT keeps the BPE segment boundaries fixed, so the comparison attributes differences to the interface rather than to boundary placement. Appendix C (Table 5) separately compares boundary sources under a fixed BBT architecture.

BBT consistently improves byte-normalized test

likelihood while using substantially fewer parameters under comparable forward FLOPs. Beyond likelihood, BBT improves completion behavior at larger hidden sizes, string-corruption robustness, and transfer to unseen languages in a multilingual setting. These results show that BBT can keep the sequence-compression benefit of BPE while avoiding the independent vocabulary rows imposed by vocabulary lookup.

2 Method

BBT is a hierarchical causal language model that keeps inputs and outputs at the byte level, using a pretrained byte-level BPE tokenizer to induce segment boundaries rather than to provide vocabulary ids for lookup or prediction. A ByteEncoder produces byte hidden states and extracts the states at segment boundaries as segment representations; the SegmentTransformer processes these representations, and the ByteDecoder uses the SegmentTransformer outputs to generate bytes (Figure 1). Thus, BBT preserves BPE’s sequence compression effect while replacing the token embedding / LM head matrix with byte-level modules.

ByteEncoder. Input bytes are embedded using a vocabulary of 256 byte values plus a few special tokens and passed through a causal ByteEncoder. Given $x = (b_1, \dots, b_T)$, we prepend BOD (beginning-of-document) and apply a sliding-window causal Transformer to obtain byte hid-

den states $H^{\text{byte}} \in \mathbb{R}^{(T+1) \times d_{\text{enc}}}$. Let e_i be the final byte position of the i -th segment defined by the BPE tokenizer. The segment representation is $h_i = H_{e_i}^{\text{byte}}$, which summarizes the segment and previous byte context without using future bytes. A trainable projection maps the BOD hidden state and $(h_1, \dots, h_N)$ from d_{enc} to d_{seg} , yielding $(\tilde{h}_0, \tilde{h}_1, \dots, \tilde{h}_N)$ as the SegmentTransformer input.

SegmentTransformer. The SegmentTransformer applies causal self-attention to $\tilde{H} = (\tilde{h}_0, \tilde{h}_1, \dots, \tilde{h}_N)$ and outputs $C = (c_0, c_1, \dots, c_N)$, where $c_i \in \mathbb{R}^{d_{\text{seg}}}$. Each c_i conditions byte decoding for the next segment s_{i+1} , while c_N predicts EOD (end of document). This module captures BBT’s long-range dependencies, with attention length proportional to the number of BPE segments N rather than the byte length T .

ByteDecoder. The ByteDecoder interleaves SegmentTransformer context vectors with teacher-forced bytes, $[c_0, b_{1,1}, \dots, b_{1,L_1}, c_1, b_{2,1}, \dots, b_{2,L_2}, \dots, c_N]$, and applies a sliding-window causal Transformer. Each c_i position predicts the first byte of the next segment; each non-final byte position predicts the following byte; the final byte of each segment predicts EOS (end of segment); and the c_N position predicts EOD. The ByteDecoder predicts only bytes and special tokens, so BBT replaces the standard BPE Transformer’s vocabulary LM head with a ByteDecoder.

Objective. The loss sums negative log-likelihood (NLL) terms for byte prediction, segment termination, and document termination:

$$\begin{aligned} \mathcal{L} = & - \sum_{i=0}^{N-1} \sum_{j=1}^{L_{i+1}} \log p(b_{i+1,j} \mid c_i, b_{i+1,<j}) \\ & - \lambda_{\text{seg}} \sum_{i=0}^{N-1} \log p(\text{EOS} \mid c_i, b_{i+1,\leq L_{i+1}}) \\ & - \log p(\text{EOD} \mid c_N) \end{aligned}$$

λ_{seg} controls the relative weight of the EOS term.

Inference. Given a prompt, the ByteEncoder encodes the observed bytes and gathers segment representations as in training. The SegmentTransformer computes contexts up to the prompt end, and the ByteDecoder generates the next segment from the final context. Within a segment, byte generation only extends the ByteDecoder prefix;

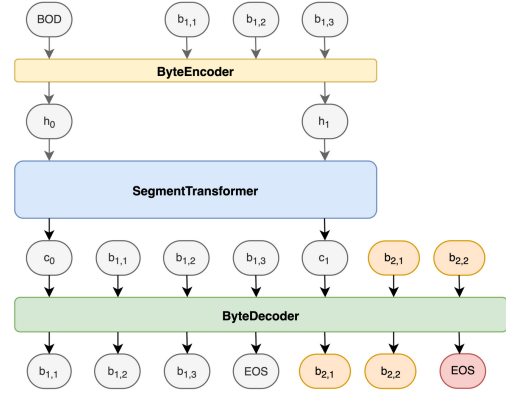


Figure 2: Inference procedure of BBT. Gray blocks denote prompt bytes, and colored blocks denote generated bytes. Within a segment, byte generation only extends the ByteDecoder prefix. When EOS is emitted, the generated byte prefix is committed as a new segment and appended to the ByteEncoder input; the ByteEncoder extracts the corresponding segment representation at the new boundary, and the SegmentTransformer uses this representation to produce the next context c_i . Generation terminates when the ByteDecoder generates EOD.

the ByteEncoder and SegmentTransformer are not recomputed (Figure 2). When EOS is emitted, the accumulated bytes are committed as a new segment and appended to the ByteEncoder input; the resulting segment representation is then passed to the SegmentTransformer to produce the next context. This repeats until EOD.

Overall, BBT uses the BPE tokenizer as segment boundary supervision rather than for vocabulary lookup, keeping BPE’s sequence compression while replacing the token embedding / LM head matrix with a ByteEncoder and a ByteDecoder.

3 Experiments

3.1 Setup

Data. Experiments use a subset of FineWeb (Penedo et al., 2024) with 47.3 GB of raw text bytes in the training split.² Validation and test documents are held out from the same source before preprocessing. We train a byte-level BPE tokenizer with a 128K vocabulary on the training split; the standard BPE Transformer uses its vocabulary ids, while BBT uses only the induced segment boundaries. The training split contains approximately 10B BPE tokens under this

²We use the Parameter Golf FineWeb export (<https://huggingface.co/datasets/willdepueoi/parameter-golf>), a derived export of FineWeb released under ODC-By v1.0 and subject to the Common Crawl Terms of Use.

tokenizer.

Matched model pairs. We compare paired standard BPE Transformer and BBT models at segment-level Transformer hidden sizes of 1024, 2048, and 3072. For each pair, the standard BPE Transformer body and BBT’s SegmentTransformer match in hidden size, layer count, attention heads, and FFN dimension. Thus, we fix the long-range causal modeling body and compare the standard BPE Transformer interface, which uses vocabulary lookup on the input side and a vocabulary LM head on the output side, with BBT’s ByteEncoder/ByteDecoder interface. The ByteEncoder and ByteDecoder use windowed self-attention with window size 128 over local byte context.

Optimization. The paired models are trained with the same data, epochs, learning-rate schedule, and optimizer settings. Matrix parameters use Muon (Liu et al., 2025), while embedding and scalar parameters use AdamW (Loshchilov and Hutter, 2019). The learning rate warms up for 200 steps and then cosine-decays to zero over the training split.

Evaluation metrics. Our main metric is BPB, the NLL of the original byte sequence divided by its byte count. For the standard BPE Transformer, token-level NLL is divided by the byte count of the original text; for BBT, BPB includes ByteDecoder NLL terms for byte targets. Thus, both models are evaluated on the likelihood they assign to the same original byte stream. We also report $\text{BPB}_{\text{eos-as-byte}}$, which adds EOS NLL and counts each EOS as one byte-like target. For auxiliary evaluations, we report LAMBADA accuracy (Paperno et al., 2016). For robustness, we report WikiText BPB and relative BPB degradation under character- and word-level corruptions on WikiText (Merity et al., 2017).

Multilingual data. For the multilingual experiments in Section 3.5, we train the paired models on a ten-language FineWeb-2 subset (Penedo et al., 2025)³ with roughly 51 GB of text in the training split. On this training split we train a separate 128K multilingual byte-level BPE tokenizer, from which the standard BPE Transformer takes vocabulary ids and BBT takes only the induced segment boundaries, as in the English setup. We evaluate the paired $d=3072$ models on FLORES-

d	Model	BPB	$\text{BPB}_{\text{eos-as-byte}}$
1024	Standard BPE	1.0200	—
	BBT	0.9924 (−2.7%)	0.9666
2048	Standard BPE	0.9690	—
	BBT	0.9272 (−4.3%)	0.9009
3072	Standard BPE	0.9447	—
	BBT	0.8823 (−6.6%)	0.8566

Table 1: Bits-per-byte (BPB) comparison on the test split. d denotes the hidden size matched between the standard BPE Transformer body and the BBT SegmentTransformer. BPB normalizes the negative log-likelihood (NLL) by the original byte count. $\text{BPB}_{\text{eos-as-byte}}$ additionally includes the end-of-segment (EOS) NLL and counts each EOS target as one byte-like unit in the denominator. Across hidden sizes 1024–3072, BBT achieves lower BPB than the standard BPE Transformer.

200 (NLLB Team et al., 2024) and report per-language BPB over the nine scripts represented in the ten-language training set, including both training languages and unseen languages written in those scripts (Table 4). All other experiments use the English FineWeb subset described above.

3.2 Byte-Level Likelihood

Table 1 shows that BBT achieves lower BPB than the standard BPE Transformer at all three matched hidden sizes, with relative reductions increasing from 2.7% at $d=1024$ to 6.6% at $d=3072$. Since each pair uses the same segment-level transformer body, this result suggests that replacing vocabulary lookup with byte-level encoder/decoder modules improves byte-level modeling performance. Even when BBT is evaluated with $\text{BPB}_{\text{eos-as-byte}}$, its score remains below the standard BPE Transformer’s BPB at every matched hidden size, so the gain remains when EOS is counted.

3.3 Parameter and Compute Accounting

We decompose each matched pair into input-side, segment-level body, and output-side components; Table 2 reports this split for both parameters and forward FLOPs/byte. The parameter columns of Table 2 show that BBT reduces parameters by replacing vocabulary-related parameters with a ByteEncoder and ByteDecoder, not by shrinking the segment-level transformer body. The FLOPs columns of Table 2 show that BBT’s interface change leaves total forward FLOPs/byte essentially unchanged: it stays within a few percent of the standard BPE Transformer at every hidden size.

³The ten languages are German, French, Russian, Greek, Arabic, Hindi, Thai, Chinese, Japanese, and Korean.

d	Component	Parameters			Forward FLOPs/byte		
		Standard BPE	BBT	Δ	Standard BPE	BBT	Δ
1024	Input side	131.1M	0.4M	−130.7M	0.0M	0.5M	+0.5M
	Segment-level body	135.3M	135.3M	+0.0M	61.2M	61.2M	+0.0M
	Output side	0 (tied)	23.1M	+23.1M	54.2M	56.0M	+1.8M
	Total	266.4M	158.8M	−40.4%	115.4M	117.7M	+1.9%
2048	Input side	262.1M	1.3M	−260.8M	0.0M	1.6M	+1.6M
	Segment-level body	721.5M	721.5M	+0.0M	316.6M	316.6M	+0.0M
	Output side	0 (tied)	39.0M	+39.0M	105.6M	86.0M	−19.6M
	Total	983.6M	761.8M	−22.5%	422.1M	404.2M	−4.2%
3072	Input side	393.2M	4.7M	−388.5M	0.0M	6.3M	+6.3M
	Segment-level body	2.819B	2.819B	+0.0M	1.212B	1.212B	+0.0M
	Output side	393.2M	55.1M	−338.1M	159.5M	120.7M	−38.8M
	Total	3.605B	2.879B	−20.2%	1.372B	1.339B	−2.4%

Table 2: Parameter and forward FLOPs/byte breakdown for paired models at matched hidden sizes. “Input side” corresponds to the token embedding in the standard BPE Transformer and to the ByteEncoder plus encoder-to-segment projection in BBT. “Segment-level body” refers to the transformer body in the standard BPE Transformer and to the SegmentTransformer in BBT. “Output side” corresponds to the vocabulary LM head in the standard BPE Transformer and to the ByteDecoder plus segment-to-decoder projection in BBT.

Appendix A gives the formulas and assumptions.

3.4 Auxiliary Evaluations

LAMBADA completion evaluation. To evaluate behavior beyond byte-level likelihood, we test the paired models on LAMBADA, which evaluates last-word completion from a passage context. Table 3 shows that BBT improves LAMBADA accuracy at the two larger hidden sizes.

WikiText corruption robustness. Table 3 also reports clean WikiText BPB and relative BPB degradation under four simple character- and word-level corruptions. Concretely, each corruption is applied at random: Char drop deletes non-space characters, Space insert adds spaces between adjacent alphabetic characters, Space drop removes existing spaces, and Shuffle permutes each word’s interior characters while keeping the first and last characters fixed. The degradation entries are relative BPB increases over clean WikiText. BBT has lower clean WikiText BPB at all three hidden sizes and lower degradation under every corruption. This supports the hypothesis that byte-level sharing improves robustness to fine-grained string perturbations.

3.5 Multilingual Generalization

Byte-level composition should help most where languages share script-level structure. Table 4 supports this: on training languages BBT lowers

macro-average BPB as in the English runs, and its gains are larger on unseen languages, especially Spanish, Bulgarian, and Marathi. These results suggest that replacing vocabulary lookup with byte-level composition helps transfer shared script-level structure to languages absent from training.

3.6 Analysis and Discussion

Hidden-size behavior. Both families improve as hidden size increases, but BBT improves more sharply: the relative BPB gain grows from 2.7% at $d=1024$ to 6.6% at $d=3072$ (Table 1), while the parameter reduction stays at least 20%. This suggests that BBT continues to benefit from added segment-level capacity while avoiding the vocabulary-sized embedding and LM head interface.

Training dynamics. Figure 3 shows how BBT’s lower BPB emerges during training. BBT starts with higher validation BPB but decreases more rapidly and crosses below the matched standard BPE Transformers by about 5 GB. This pattern indicates that the initial BPB gap is transient rather than evidence of an architectural capability difference; Appendix B explains its cause.

Possible role of byte-level sharing. BBT and the standard BPE Transformer use the same BPE tokenizer to define segment boundaries, but BBT does not treat each segment as an opaque vocabulary id. Instead, BBT computes each segment representation from its byte sequence and local context, rather

	$d=1024$		$d=2048$		$d=3072$	
Metric	Standard BPE	BBT	Standard BPE	BBT	Standard BPE	BBT
<i>LAMBADA accuracy ($\uparrow$)</i>						
LAMBADA	26.70	26.00	30.02	33.81	30.20	38.17
<i>WikiText robustness</i>						
Clean BPB ($\downarrow$)	1.1859	1.1700	1.1263	1.0957	1.1013	1.0483
<i>Relative degradation over clean BPB ($\downarrow$)</i>						
Char drop	114.5%	78.5%	119.3%	81.7%	121.4%	83.6%
Space insert	147.8%	98.1%	151.2%	95.0%	153.7%	89.8%
Space drop	34.5%	31.6%	34.5%	29.6%	34.5%	28.7%
Shuffle	210.8%	167.4%	223.3%	177.2%	228.8%	184.5%

Table 3: LAMBADA completion accuracy and WikiText robustness for the paired models. Clean BPB is clean WikiText BPB; the four corruption rows report relative BPB degradation percentages over clean WikiText.

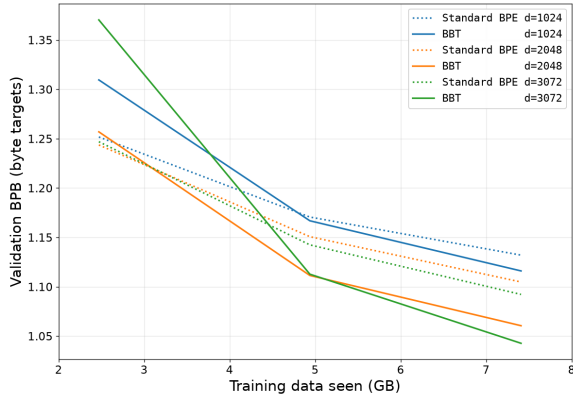


Figure 3: Early portion of the validation BPB learning curves. Colors denote matched hidden size, and line styles denote model family (dotted=standard BPE Transformer, solid=BBT). BBT starts from a higher early-training BPB, but catches up sooner at larger hidden sizes. Later in training, BBT maintains lower validation BPB than the corresponding standard BPE Transformer at every matched hidden size.

than assigning an independent vector to each token id. Segments sharing prefixes, suffixes, spelling patterns, punctuation, or whitespace patterns therefore share parameters through the ByteEncoder and ByteDecoder. In the standard BPE Transformer, similar segments still occupy separate vocabulary rows, making rare tokens harder to learn through independent embedding and LM head rows. This provides a plausible explanation for BBT’s lower BPB, smaller corruption degradation, and stronger transfer to unseen languages written in scripts covered during training.

Segmentation as a separable design choice. BBT suggests that BPE segmentation and vocab-

ulary lookup should be treated as separate design choices. Our results show that BPE segmentation remains useful as a compression mechanism for long-range modeling, even when token ids are never used for input vocabulary lookup or as the output LM head prediction space. This separation is especially important for lower-capacity language models, where the token embedding / LM head matrix can consume a substantial fraction of the parameter budget.

4 Related Work

Subword tokenization and embedding cost. BPE (Sennrich et al., 2016) and its variants are the standard subword tokenization methods for modern LLMs. Larger vocabulary sizes can improve multilingual/code coverage and shorten sequences (Dagan et al., 2024; Tao et al., 2024), but they also make the token embedding / LM head matrix large, as in LLaMA 3’s 128K vocabulary (Grattafiori et al., 2024; Meta AI, 2024). This cost is most visible in lower-capacity models, where vocabulary lookup can consume capacity that could otherwise go to the segment-level transformer body. BBT addresses this by keeping BPE segment boundaries while removing vocabulary lookup.

Tokenizer-free byte / character models. ByT5 (Xue et al., 2022), Charformer (Tay et al., 2022), CANINE (Clark et al., 2022), and related models remove the tokenizer by using bytes or characters. They improve open-vocabulary and multilingual robustness, but make sequences roughly 4–5 $\times$ longer, increasing self-attention cost. BBT also uses byte-level inputs and outputs,

	Language (script)	Standard BPE	BBT	$\Delta\%$
Training langs.	German (Latin)	1.1246	1.1097	-1.3
	French (Latin)	1.0326	1.0094	-2.2
	Russian (Cyrillic)	0.7010	0.6817	-2.7
	Greek (Greek)	0.6512	0.6447	-1.0
	Arabic (Arabic)	0.9171	0.8821	-3.8
	Hindi (Devanagari)	0.6118	0.5639	-7.8
	Thai (Thai)	0.5391	0.4899	-9.1
	Chinese (Han)	1.4963	1.5076	+0.8
	Japanese (Han+Kana)	1.0829	1.0546	-2.6
	Korean (Hangul)	1.1835	1.1597	-2.0
	<i>macro-avg</i>	0.9340	0.9104	-2.5
Unseen langs.	English (Latin)	1.5364	1.4514	-5.5
	Spanish (Latin)	2.0608	1.7889	-13.2
	Ukrainian (Cyrillic)	1.5139	1.4057	-7.1
	Bulgarian (Cyrillic)	1.8651	1.5964	-14.4
	Persian (Arabic)	2.1463	2.1439	-0.1
	Marathi (Devanagari)	1.2620	1.1337	-10.2
	<i>macro-avg</i>	1.7308	1.5867	-8.3

Table 4: Held-out per-language BPB on FLORES-200 for the paired multilingual comparison at $d=3072$. Rows are grouped by whether the language appears in the training mix; unseen languages are written in scripts covered during training. $\Delta\%$ is BBT relative to the standard BPE Transformer (negative favors BBT). BBT improves macro-average BPB by 2.5% on training languages, and generalizes better to unseen languages written in scripts covered during training, where it lowers macro-average BPB by 8.3%.

but applies global self-attention to the BPE-defined segment sequence.

Hierarchical byte transformers. MegaByte (Yu et al., 2023) uses fixed-length byte patches in a local-global 2-tier architecture, while SpaceByte (Slagle, 2024) uses surface-form heuristics such as whitespace. Byte Latent Transformer (BLT) (Pagnoni et al., 2024) sets patch boundaries from byte-level uncertainty predicted by an entropy model. ByteFlow (Deng et al., 2026) replaces the tokenizer with an information-theoretic byte chunker based on coding-rate compression. These methods differ mainly in how they choose segment boundaries. BBT instead reuses the pretrained BPE merge table as a corpus-adaptive boundary source, inheriting segment boundaries from the tokenizer rather than introducing a separate patching rule or boundary-selection mechanism. Unlike MegaByte/SpaceByte/BLT/ByteFlow, which are tokenizer-free, BBT keeps BPE segmentation but removes vocabulary lookup; Appendix C reports a controlled boundary-source ablation under a BBT architecture. BBT therefore occupies a middle ground: it keeps BPE-derived segment bound-

aries while replacing vocabulary lookup and LM head prediction with ByteEncoder/ByteDecoder composition.

5 Conclusion

This paper introduced BPE-Guided Byte Transformer (BBT), which separates BPE segmentation from vocabulary lookup. Instead of using vocabulary lookup or LM head, BBT uses BPE segment boundaries to gather byte-level hidden states, performs segment-level causal modeling, and decodes the next segment at the byte level.

On a subset of FineWeb, BBT achieved lower BPB than the standard BPE Transformer at every matched hidden size while reducing total parameters by 20.2–40.4% and keeping the same segment-level transformer body, with comparable forward FLOPs/byte. Beyond BPB, auxiliary evaluations showed that BBT improves LAMBADA completion at larger hidden sizes and robustness to fine-grained string corruptions. A paired multilingual comparison further showed that BBT improves held-out BPB on training languages and generalizes better to unseen languages written in scripts covered during training.

By preserving BPE’s sequence compression while replacing vocabulary lookup with a ByteEncoder/ByteDecoder interface, BBT is a candidate for pretraining regimes where the token embedding / LM head matrix would consume a large share of the parameter budget.

Limitations

Our experiments are limited to English FineWeb and a ten-language subset of FineWeb-2. Although BBT achieved lower BPB than the standard BPE Transformer with fewer parameters, and the multilingual run further showed better cross-lingual generalization to unseen languages written in scripts covered during training, further validation is needed on larger datasets, larger models, a broader set of languages and scripts, and code corpora, where vocabulary-related parameters may matter more.

BBT also has a more complex architecture and inference procedure than a standard BPE Transformer. It requires separate ByteEncoder, SegmentTransformer, and ByteDecoder modules, and introduces additional hyperparameters such as local attention window size, encoder/decoder depth, and EOS loss weight. Practical deployment must weigh

this complexity against parameter and FLOPs savings.

Finally, although we complement BPB with LAMBADA and WikiText robustness evaluations, these still do not capture long-form generation quality. Assessing BBT’s full practical value therefore requires further generation-quality and efficiency evaluation.

References

- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. [Enriching word vectors with subword information](#). *Transactions of the Association for Computational Linguistics*, 5:135–146.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. [CANINE: Pre-training an efficient tokenization-free encoder for language representation](#). *Transactions of the Association for Computational Linguistics*, 10:73–91.
- Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. 2024. [Getting the most out of your tokenizer for pre-training and domain adaptation](#). *Preprint*, arXiv:2402.01035.
- Chunyan Deng, Sanket Lokegaonkar, Colin Lockard, Besnik Fetahu, Nasser Zalmout, and Xian Li. 2026. [ByteFlow: Language modeling through adaptive byte compression without a tokenizer](#). *Preprint*, arXiv:2603.03583.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Taku Kudo and John Richardson. 2018. [SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium. Association for Computational Linguistics.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, and 9 others. 2025. [Muon is scalable for LLM training](#). *Preprint*, arXiv:2502.16982.
- Ilya Loshchilov and Frank Hutter. 2019. [Decoupled weight decay regularization](#). In *International Conference on Learning Representations*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *International Conference on Learning Representations*.
- Meta AI. 2024. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md. Accessed: 2026-05-29.
- Pit Neiteimeier, Björn Deiseroth, Constantin Eichenberg, and Lukas Balles. 2025. [Hierarchical autoregressive transformers: Combining byte- and word-level processing for robust, adaptable language models](#). *Preprint*, arXiv:2501.10322.
- NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Hefernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, and 1 others. 2024. [Scaling neural machine translation to 200 languages](#). *Nature*, 630:841–846.
- Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. 2024. [Byte latent transformer: Patches scale better than tokens](#). *Preprint*, arXiv:2412.09871.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. [The LAMBADA dataset: Word prediction requiring a broad discourse context](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1525–1534.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben al-lal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2024. [The FineWeb datasets: Decanting the web for the finest text data at scale](#). *Preprint*, arXiv:2406.17557.
- Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. 2025. [FineWeb2: One pipeline to scale them all – adapting pre-training data processing to every language](#). *Preprint*, arXiv:2506.20920.
- Yuval Pinter, Robert Guthrie, and Jacob Eisenstein. 2017. [Mimicking word embeddings using subword RNNs](#). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 102–112, Copenhagen, Denmark. Association for Computational Linguistics.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.
- Kevin Slagle. 2024. [SpaceByte: Towards deleting tokenization from large language modeling](#). *Preprint*, arXiv:2404.14408.

- Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muenighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. 2024. [Scaling laws with vocabulary: Larger models deserve larger vocabularies](#). *Preprint*, arXiv:2407.13623.
- Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. 2022. [CharFormer: Fast character transformers via gradient-based subword tokenization](#). In *International Conference on Learning Representations*.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.
- Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. [MEGABYTE: Predicting million-byte sequences with multiscale transformers](#). *Preprint*, arXiv:2305.07185.

A FLOPs Calculation

FLOPs/byte denotes major forward-pass matmul FLOPs divided by the number of original bytes. The FLOPs columns of Table 2 report forward FLOPs/byte.

Let F be the per-position major matmul FLOPs for a Transformer block stack. For hidden size d , h_{kv} KV heads, head dimension d_h , L layers, effective attention length a , and SwiGLU width d_{ff} ,

$$F = L (4d^2 + 4dh_{kv}d_h + 4ad + 6dd_{ff}).$$

The terms are Q/O projections, GQA K/V projections, attention score/value matmuls, and SwiGLU FFN matmuls.

Standard BPE Transformer. Let B , T_{tok} , N_{byte} , and V_{BPE} denote batch size, maximum token length, original-byte count, and BPE vocabulary size. Since the standard BPE Transformer uses full causal token attention, $a_{\text{base}} = T_{\text{tok}}$. With per-position FLOPs F_{base} ,

$$C_{\text{base}} = BT_{\text{tok}} (F_{\text{base}} + 2dV_{\text{BPE}}).$$

The second term is the vocabulary LM head cost; we report $C_{\text{base}}/N_{\text{byte}}$.

BBT. For BBT, let B denote batch size, and let T_{byte} and T_{seg} be the maximum byte and segment sequence lengths. The ByteEncoder and ByteDecoder use windowed causal attention; the SegmentTransformer uses full causal attention over segments. The decoder length is

$$T_{\text{dec}} = T_{\text{byte}} + T_{\text{seg}}$$

With ByteEncoder and ByteDecoder windows W_{enc} and W_{dec} ,

$$a_{\text{enc}} = \min(W_{\text{enc}}, T_{\text{byte}}),$$

$$a_{\text{seg}} = T_{\text{seg}},$$

$$a_{\text{dec}} = \min(W_{\text{dec}}, T_{\text{dec}})$$

Let F_{enc} , F_{seg} , and F_{dec} be the corresponding per-position FLOPs. Then

$$\begin{aligned} C_{\text{BBT}} = & BT_{\text{byte}}F_{\text{enc}} \\ & + BT_{\text{seg}}\mathbf{1}_{d_{\text{enc}} \neq d_{\text{seg}}}2d_{\text{enc}}d_{\text{seg}} \\ & + BT_{\text{seg}}F_{\text{seg}} \\ & + BT_{\text{dec}}F_{\text{dec}} \\ & + BT_{\text{seg}}\mathbf{1}_{d_{\text{seg}} \neq d_{\text{dec}}}2d_{\text{seg}}d_{\text{dec}} \\ & + BT_{\text{dec}}2d_{\text{dec}}V_{\text{out}}. \end{aligned}$$

The projection terms are encoder-to-segment and segment-to-decoder costs. The final term is the byte-level LM head with $V_{\text{out}} = 260$; we report $C_{\text{BBT}}/N_{\text{byte}}$.

B Initial BPB Gap

In the first validation interval of Figure 3, BBT starts with higher BPB than the paired standard BPE Transformer because the two interfaces predict different units under byte-normalized BPB. The standard BPE Transformer predicts BPE-compressed targets with a 128K-class vocabulary softmax; near-uniform initialization assigns about $\log_2 |\mathcal{V}_{\text{BPE}}|$ bits to one symbol, amortized over its average byte length:

$$\text{BPB}_{\text{BPE}} \approx \frac{\log_2 |\mathcal{V}_{\text{BPE}}|}{\mathbb{E}[\ell_{\text{seg}}]}$$

BBT predicts byte targets with an output vocabulary of 256 byte values together with four special tokens (BOD, EOS, EOD, and PAD). Near-uniform initialization assigns about $\log_2 |\mathcal{V}_{\text{out}}|$ bits to each byte target, and because both the total loss and denominator scale with the number of bytes,

$$\text{BPB}_{\text{BBT}} \approx \frac{\mathbb{E}[\ell_{\text{seg}}] \log_2 |\mathcal{V}_{\text{out}}|}{\mathbb{E}[\ell_{\text{seg}}]} = \log_2 |\mathcal{V}_{\text{out}}|.$$

The early gap is therefore interface-level: the standard BPE Transformer makes one softmax decision for a multi-byte segment, whereas BBT pays per byte. For a 7-byte BPE token with $|\mathcal{V}_{\text{BPE}}| = 128,000$, the standard BPE Transformer starts near $\log_2(128,000)/7 \approx 2.42$ BPB; with $|\mathcal{V}_{\text{out}}| = 260$, BBT starts near $\log_2(260) \approx 8.02$ BPB. As training proceeds, BBT learns byte-level composition and segment termination and later maintains lower validation BPB.

C Segmentation Alternatives

BBT must choose which boundaries the global SegmentTransformer processes. In addition to BPE-guided segmentation, we test fixed-length byte patches and SpaceByte-style spacelike boundaries with the same $d=3072$ BBT configuration; Table 5 reports test BPB. Fixed- p segmentation directly controls segment length. Choosing p near the average byte length of the 128K BPE tokenizer roughly matches the segment sequence length of BPE-guided segmentation. However, fixed boundaries can split morphemes, words, UTF-8 multibyte characters, or frequent byte patterns, so the units

Segmentation	Boundary source	#Params	BPB
Fixed- p byte ($p=4$)	Every 4 bytes	2.879B	0.9764
Fixed- p byte ($p=6$)	Every 6 bytes	2.879B	1.0283
SpaceByte-style	Spacelike UTF-8 bytes	2.879B	0.9175
BPE-guided	BPE merge table	2.879B	0.8823

Table 5: Boundary-source ablation for large-scale BBT. All rows use the same architecture and data splits; BPB is measured on the test split.

are not corpus-adaptive; Table 5 shows higher BPB than BPE-guided segmentation.

SpaceByte-style segmentation places a boundary after bytes classified as spacelike by a UTF-8 heuristic. This uses surface cues such as spaces, punctuation, control characters, and UTF-8 leading bytes, while keeping ASCII letters/digits and continuation bytes inside segments. It is more text-aware than fixed- p , but the hand-written rule may not fit code or multilingual text; it beats fixed- p but remains worse than BPE-guided segmentation under the same architecture and training budget.

D Experimental Details

Table 6 reports the architecture settings for each standard BPE Transformer–BBT model pair. At each hidden size, the standard BPE Transformer body and the BBT SegmentTransformer share the same hidden size, layer count, attention heads, and FFN dimension. Table 7 reports the shared data and optimizer settings. All models were trained on a single node with 8 NVIDIA H200 GPUs, where a single training run of the largest model ($d=3072$) takes about 65 wall-clock hours.

	Standard BPE			BBT		
Hidden size	1024	2048	3072	1024	2048	3072
Vocabulary	128K	128K	128K	260	260	260
Tied LM head	Yes	Yes	No	No	No	No
<i>ByteEncoder</i>						
d_{enc}	–	–	–	128	256	512
Layers	–	–	–	1	1	1
heads / KV heads	–	–	–	16/4	16/4	16/4
Attention window	–	–	–	128	128	128
<i>Segment-level transformer body</i>						
$d_{\text{model}} / d_{\text{seg}}$	1024	2048	3072	1024	2048	3072
Layers	12	16	28	12	16	28
heads / KV heads	16/4	32/8	24/8	16/4	32/8	24/8
FFN dim	2816	5632	8192	2816	5632	8192
<i>ByteDecoder</i>						
d_{dec}	–	–	–	1024	1280	1536
Layers	–	–	–	2	2	2
heads / KV heads	–	–	–	16/4	16/4	16/4
Attention window	–	–	–	128	128	128
EOS loss weight	–	–	–	0.05	0.05	0.05

Table 6: Architecture hyperparameters across matched hidden sizes. The standard BPE Transformer body and BBT SegmentTransformer share the same hidden size, number of layers, FFN dimension, RoPE base, RMSNorm setting, initialization std, and GQA ratio within each pair. The main differences are that the standard BPE Transformer’s 128K token embedding and vocabulary LM head are replaced in BBT by a small ByteEncoder and a separate ByteDecoder. All models use RMSNorm $\epsilon = 10^{-5}$ and initialization std = 0.02.

Settings	Value		
<i>Training setup</i>			
Bytes (train/val/test)	47.3 GB / 15.0 MB / 15.0 MB		
Max tokens/segments per sample	1,024		
Max bytes per sample	8,192		
Global batch size	512		
Max epochs	1		
<i>Optimizer</i>	<i>d=1024</i>	<i>d=2048</i>	<i>d=3072</i>
Embedding/head LR	0.03 / –	0.02 / –	0.015 / 0.015
Matrix LR	0.002	0.0015	0.001
Scalar LR	0.002	0.0015	0.001
Muon momentum	0.95		
Muon Nesterov	Yes		
Muon NS steps	5		
AdamW params	$\beta_1 = .9, \beta_2 = .95, \epsilon = 10^{-8}$		

Table 7: Training and optimizer settings. Within each hidden size, the standard BPE Transformer and BBT share the same data, schedule, batch size, optimizer type, and learning rates. Max tokens/segments per sample denotes tokens for the standard BPE Transformer and segments for BBT.